Projet de génie logiciel

Jeu d'aventures

INFO3 — Polytech Grenoble — 2026-2027

Auteurs : à compléter

L'application est un jeu d'exploration. Le joueur se déplace de salle en salle, ramasse des objets, parle à un personnage, ouvre une porte et doit atteindre un trésor. La fenêtre montre l'image de la salle, un texte qui décrit le lieu et les actions possibles, puis des boutons pour ces actions.

1. Document d'analyse

1.1 Acteurs

Le seul acteur est le joueur. C'est lui qui lance la partie et qui choisit chaque action. L'ermite et le garde ne sont pas des acteurs : ils ne prennent aucune initiative. Ils font partie du système et réagissent quand le joueur leur parle ou tente de passer.

1.2 Diagramme de cas d'utilisation

Les cas suivent les actions demandées par le sujet : se déplacer, observer, ramasser, transporter (le sac), utiliser, rencontrer un personnage et lui parler. Quitter ferme la fenêtre. Trouver le trésor n'est pas un bouton : c'est le résultat d'un déplacement dans la salle du trésor.

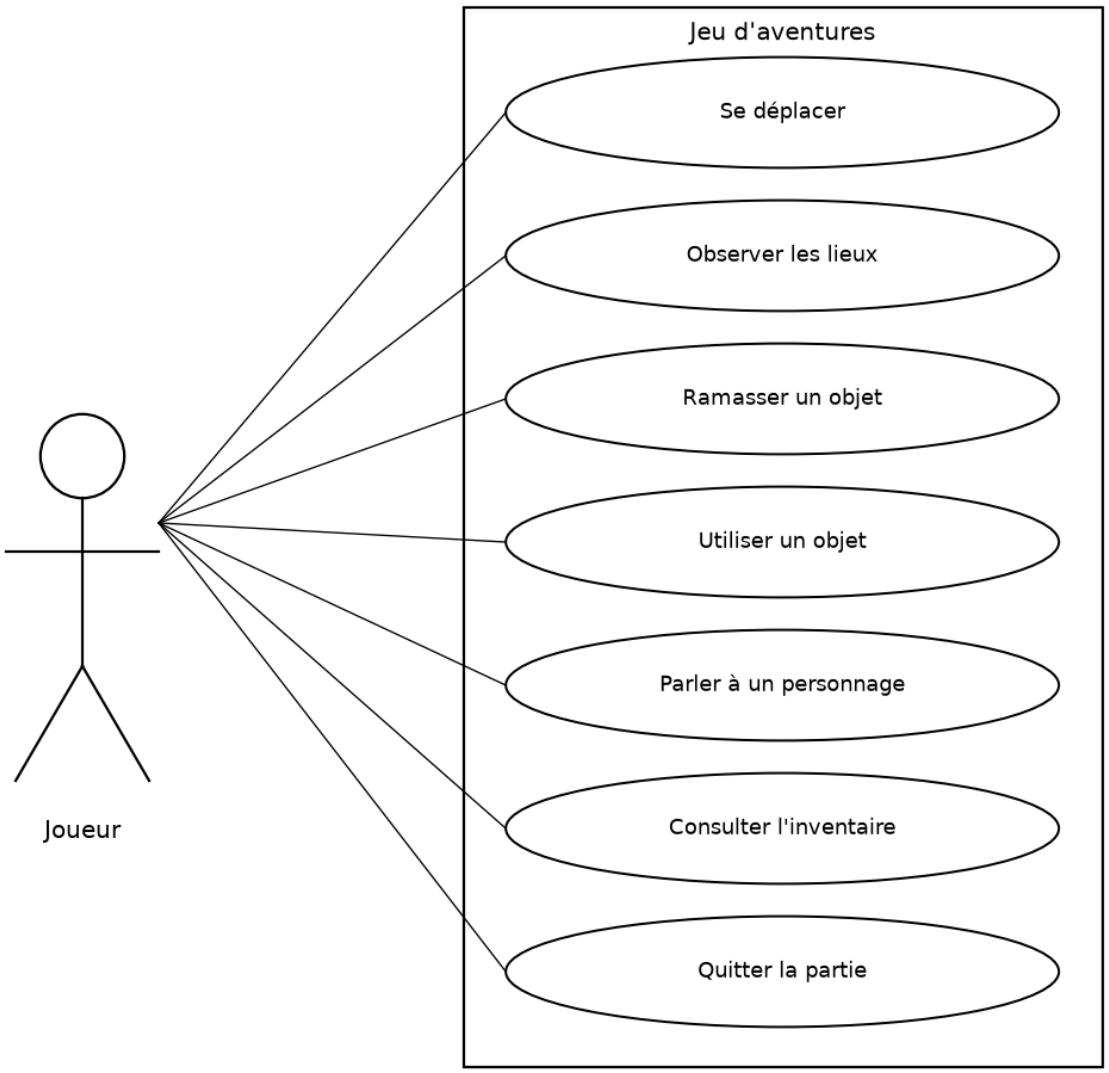


Figure 1. Cas d'utilisation. L'acteur Joueur est à l'extérieur du système.

1.3 Description des principaux cas d'utilisation

Se déplacer. Le joueur choisit une direction proposée pour la salle courante. Scénario nominal : la direction existe, la porte n'est pas verrouillée, personne ne bloque, et le joueur entre dans la salle voisine. La fenêtre change d'image et de texte. Alternatives : pas de passage dans cette direction ; la porte est verrouillée ; le garde bloque encore la sortie (s'il n'a pas d'épée, la partie est perdue ; s'il a l'épée mais ne l'a pas utilisée, le jeu lui demande de le faire).

Ramasser un objet. Le joueur choisit un objet posé dans la salle. Scénario nominal : l'objet est transportable et le sac n'est pas trop lourd (poids maximum 5 ; la clé pèse 1, l'épée pèse 2). L'objet quitte la salle et entre dans le sac. Alternatives : l'objet n'est pas transportable (le trésor) ; le sac est trop lourd ; l'objet n'est plus dans la salle.

Utiliser un objet. Le joueur choisit un objet de son sac. Deux utilisations sont utiles. Dans le couloir, l'épée fait reculer le garde. Toujours dans le couloir, la clé rouillée ouvre la porte de l'est. Ailleurs, le jeu répond que l'objet ne sert à rien ici. Une porte déjà ouverte le reste.

Parler. S'il n'y a personne, le jeu le dit. L'ermite donne l'épée une seule fois. S'il n'a plus rien, il le dit. Le garde refuse de parler. Observer répète la description de la salle et ce qui est au sol. Consulter l'inventaire écrit le contenu du sac dans le message sous le texte. Le sac reste aussi affiché en permanence sous ce message.

1.4 Diagramme de séquence système

Le scénario nominal pour gagner enchaîne plusieurs cas : se déplacer, ramasser la clé, parler à l'ermite, utiliser l'épée, utiliser la clé, puis entrer dans le trésor. Le diagramme suivant reste au niveau du système : on ne voit pas encore les classes. Les flèches en pointillés sont les réponses du système.

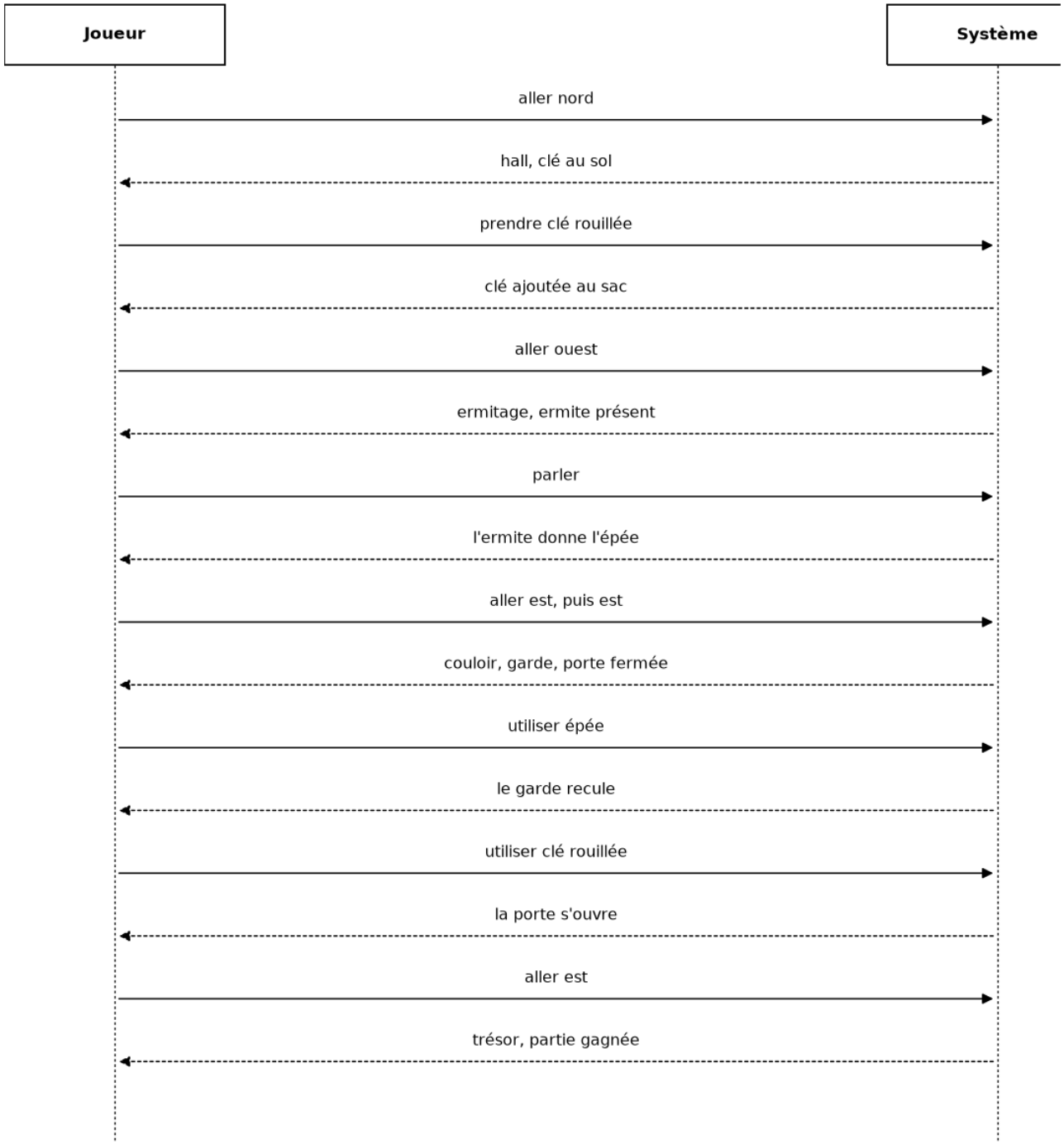


Figure 2. Séquence système du scénario qui mène au trésor.

Une alternative importante, au moment « aller est » dans le couloir : si le joueur n'a pas l'épée, le système termine la partie sur une défaite et n'ouvre pas la porte.

1.5 Diagramme de classes d'analyse

Le monde est un graphe de salles. Une sortie relie une salle à une autre. Elle peut exiger un objet (porte verrouillée) et être gardée. Un personnage est dans au plus une salle et peut offrir un objet. Le joueur a une salle courante et un sac. La partie mémorise si elle est terminée et gagnée. Les noms du diagramme sont ceux du domaine. Les classes du code, en anglais, sont données dans la conception.

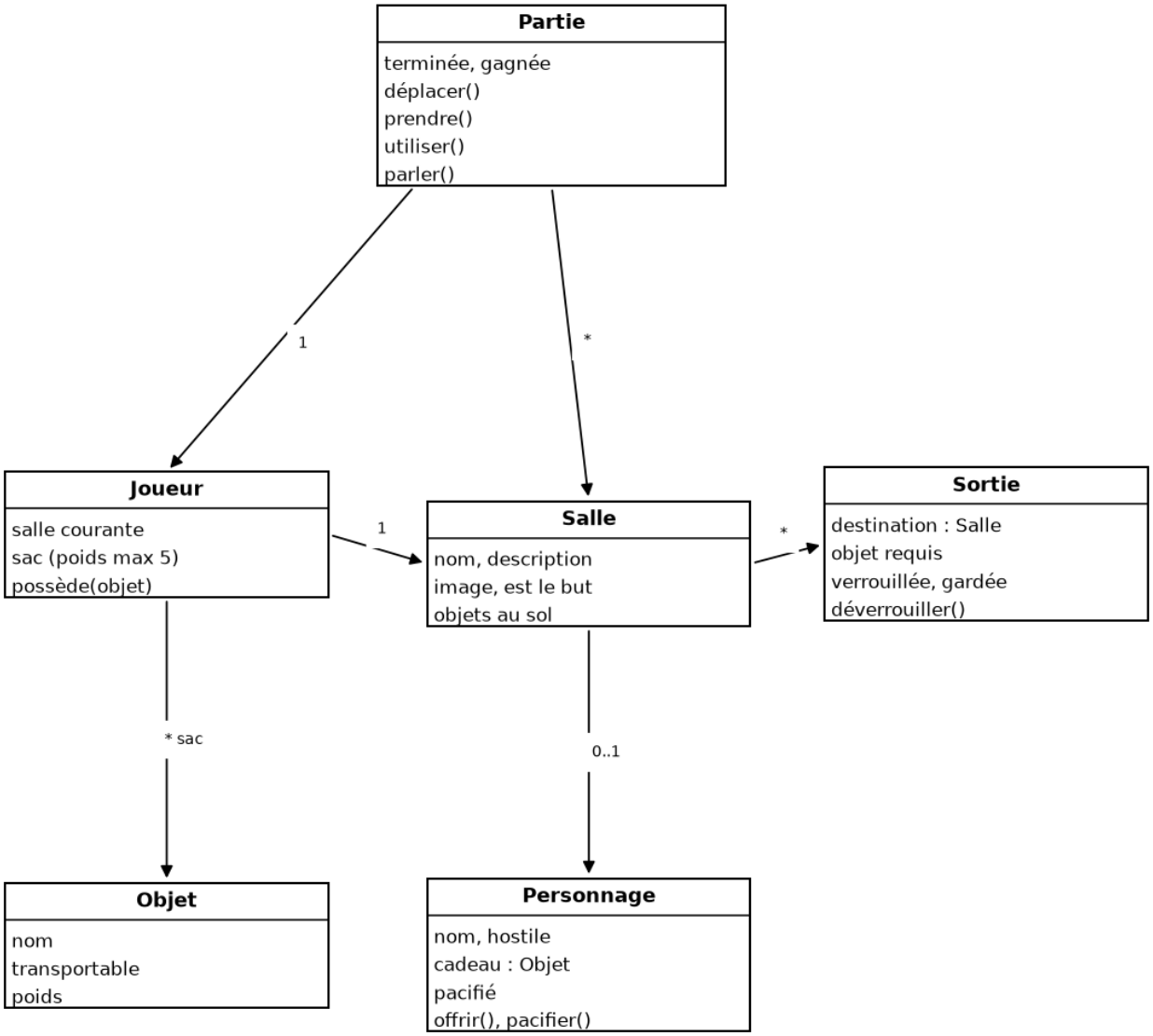


Figure 3. Classes d'analyse et leurs liens.

2. Document de conception

2.1 Architecture MVC

Le modèle contient les règles et ne fait aucun affichage. Il n'importe ni Swing ni le contrôleur. La vue est la fenêtre Swing. Le contrôleur est le seul objet qui modifie le modèle puis demande à la vue de se redessiner.

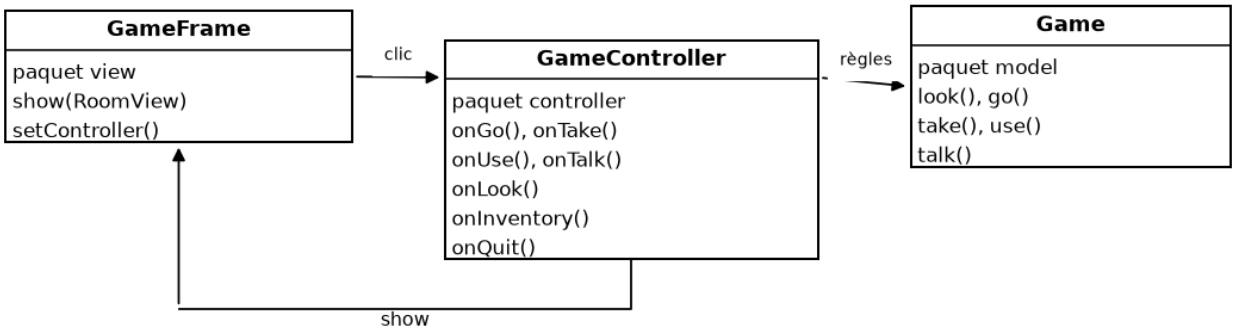
Modèle : Item, Exit, Personnage, Room, Player, Game, RoomView. RoomView est une copie des informations à afficher (nom, image, sorties, objets, sac, message, partie terminée). La fenêtre ne parcourt pas les salles elle-même.

Vue : GameFrame. Elle affiche l'image, le texte (salle, description, personnage, objets, liste des actions possibles), le message, le sac, puis les boutons. Elle ne décide pas si une porte s'ouvre.

Contrôleur : GameController. Chaque bouton appelle une méthode (onGo, onTake,

onUse, onTalk, onLook, onInventory, onQuit). Cette méthode appelle Game, puis show avec le résultat de look(). Main crée la partie, la fenêtre et le contrôleur, puis appelle start().

La vue connaît le contrôleur, parce que les boutons l'appellent. Le contrôleur connaît la vue et le modèle. Le modèle ne connaît ni l'un ni l'autre. On peut donc changer la fenêtre sans réécrire les règles. Les tests de Game (mvn test) s'exécutent d'ailleurs sans ouvrir de fenêtre.



GameFrame appelle le contrôleur. Le contrôleur appelle Game, puis GameFrame.show. Game ne voit pas la fenêtre.

Figure 4. Classes de l'architecture et le sens des appels.

2.2 Diagramme de classes logicielles

La classe s'appelle Personnage et non Character, pour ne pas la confondre avec java.lang.Character. Game ne garde pas de liste de salles : createWorld relie les salles entre elles, et le joueur retient la salle courante. Le reste du graphe reste en mémoire parce que chaque sortie pointe vers une salle.

Carte construite dans Game.createWorld : Entrée (nord) Grand hall ; Grand hall (ouest) Ermitage, (est) Couloir, (sud) Entrée ; Couloir (ouest) Grand hall, (est) Salle du trésor. Cette dernière sortie est gardée et verrouillée par la clé rouillée. La clé est au sol dans le hall. L'épée est le cadeau de l'ermite. Le trésor est dans la salle du but, il n'est pas transportable. Entrer dans cette salle met finished et won à vrai.

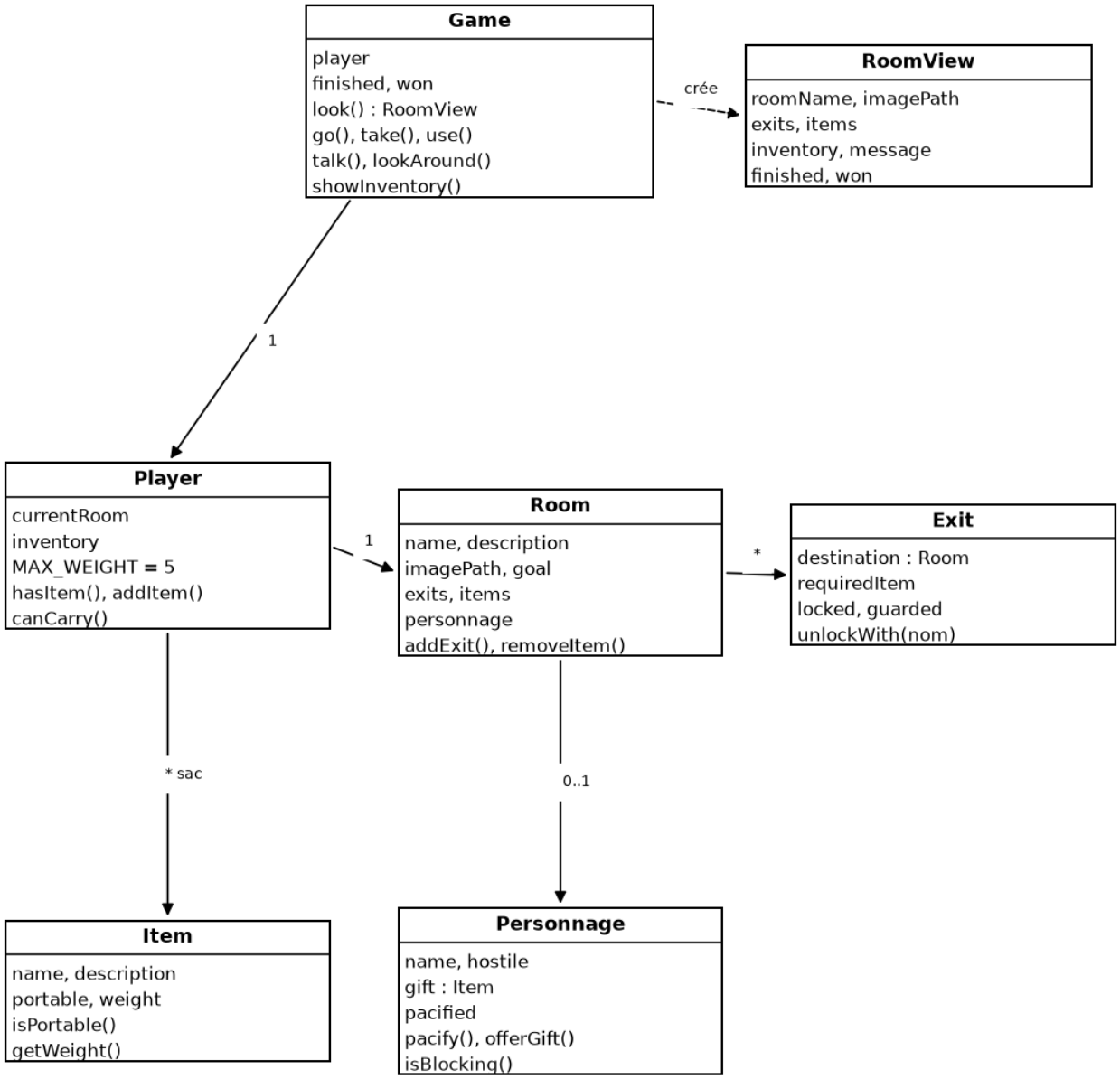


Figure 5. Classes du modèle. La flèche en pointillés signifie que Game fabrique un RoomView.

2.3 Diagramme de séquence de conception

Le diagramme suit un clic sur « Prendre clé rouillée », depuis le bouton jusqu'au rafraîchissement de la fenêtre. take retire l'objet de la salle courante et, s'il est transportable et pas trop lourd, l'ajoute au sac du joueur. look construit ensuite le RoomView que la vue affiche. Les autres boutons suivent le même chemin : une méthode de GameController, une méthode de Game, puis show.

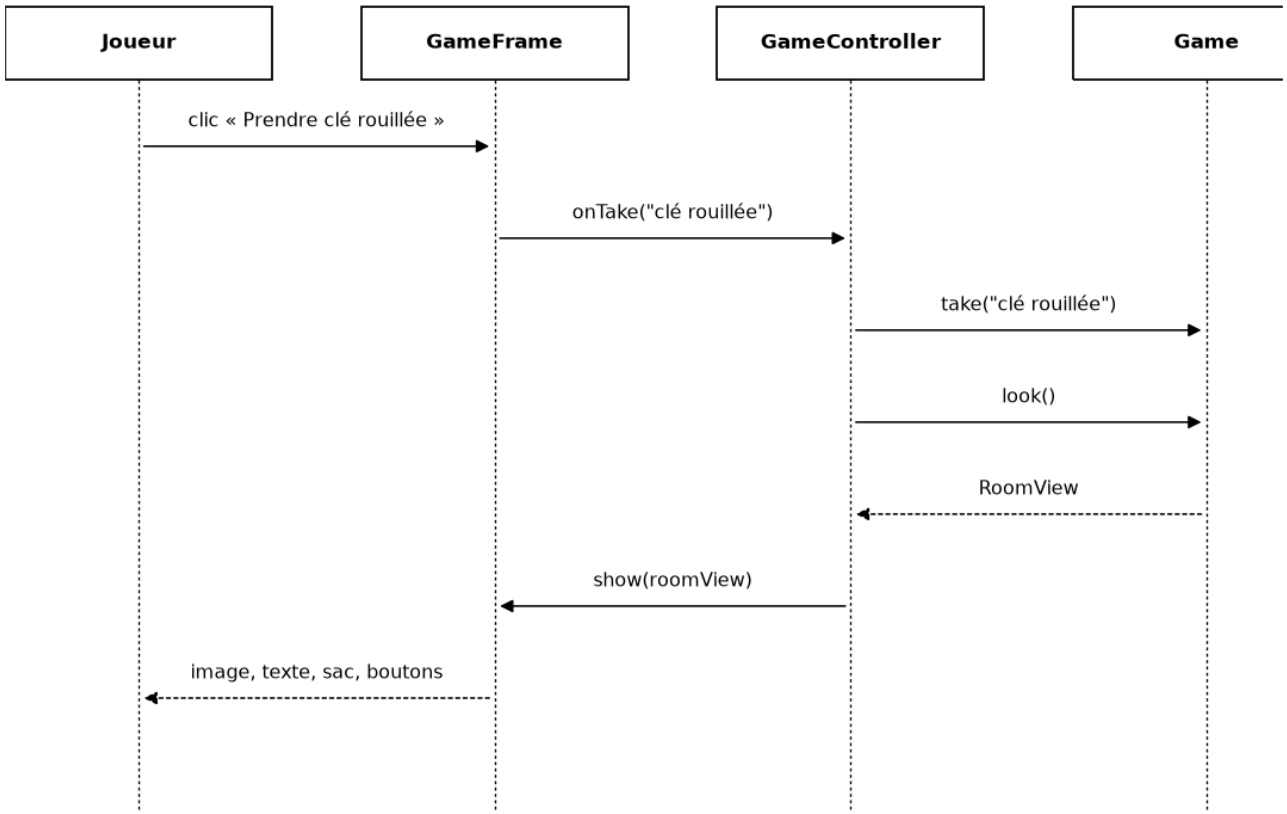


Figure 6. Séquence MVC pour ramasser la clé. Les retours sont en pointillés.

2.4 Diagrammes d'états-transitions

Trois états méritent un diagramme. La partie commence « en cours ». Elle devient gagnée si le joueur entre dans la salle dont le but est vrai, et perdue s'il force la sortie gardée sans épée. La porte du trésor passe de verrouillée à ouverte quand la clé est utilisée dans le couloir. Le garde passe de « bloque » à « laisse passer » quand l'épée est utilisée dans le couloir. Tant qu'il bloque, une tentative de passage avec l'épée dans le sac ne change pas son état : le joueur doit utiliser l'objet.

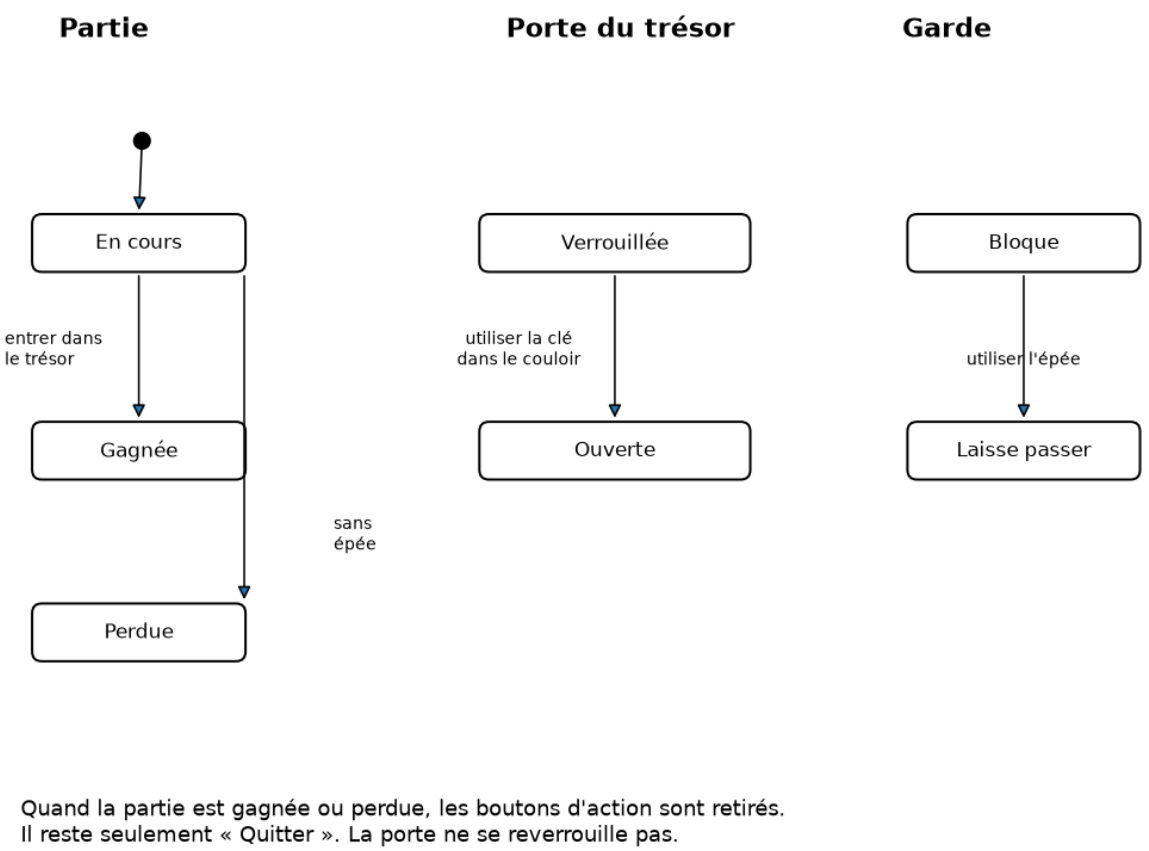


Figure 7. États de la partie, de la porte et du garde.

3. Manuel utilisateur

3.1 Lancer le jeu

Il faut un JDK 17 (la commande javac) et Maven. Dans le dossier qui contient pom.xml :

mvn -q compile exec:java

Une fenêtre « Jeu d'aventures » s'ouvre. Rien à configurer, pas de mot de passe.

3.2 La fenêtre

En haut, l'image de la salle. Au milieu, le nom de la salle, sa description, qui est présent, ce qui est au sol, et la liste « Vous pouvez ». Sous ce texte, un message en gras indique le résultat de la dernière action. En dessous, le sac (poids actuel / 5). Tout en bas, les boutons. Ils reprennent exactement les actions listées dans le texte.

Aller nord, Aller sud, Aller est, Aller ouest : changer de salle quand la direction existe. Prendre … : ramasser l'objet nommé. Utiliser … : utiliser un objet du sac. Parler : parler au personnage de la salle. Observer : relire la salle. Inventaire : rappeler le contenu du sac dans le message. Quitter : fermer la fenêtre.

3.3 Carte et partie

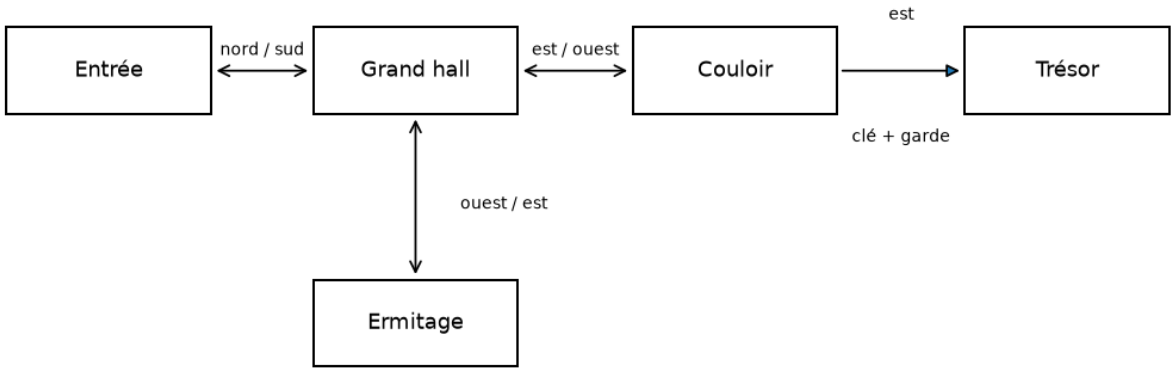


Figure 8. Carte. La flèche vers le trésor est à sens unique dans la pratique : entrer gagne la partie.

Pour gagner : Aller nord (hall), Prendre clé rouillée, Aller ouest (ermitage), Parler (l'ermite donne l'épée), Aller est puis Aller est (couloir), Utiliser épée, Utiliser clé rouillée, Aller est. Le message « Vous avez trouvé le trésor. Vous avez gagné. » s'affiche et les boutons d'action disparaissent.

Pour perdre : entrer dans le couloir et choisir Aller est sans avoir l'épée. Le garde frappe et la partie s'arrête. Avec l'épée dans le sac, Aller est ne fait que demander d'utiliser l'épée. La clé utilisée hors du couloir ne fait rien. Le trésor s'affiche dans la salle finale mais ne peut pas être emporté.

4. Bilan

Le programme est en Java 17, construit avec Maven, affiché avec Swing (inclus dans le JDK). Les règles sont vérifiées par JUnit, sans fenêtre. Les diagrammes de ce rapport ont été dessinés pour nommer les mêmes classes et les mêmes méthodes que le code.

La difficulté principale était de ne pas mélanger l'affichage et les règles. Un exemple de jeu texte, World of Zuul (Barnes et Kölling, Objects First with Java), est présent à la racine du dépôt. Il n'a pas été repris : la boucle de jeu et les affichages y sont dans la même classe, et cette version ne compile pas en l'état. Le sujet demande déjà des salles reliées entre elles. Le projet Maven repart de cette idée, avec un modèle qui ne fait que renvoyer un RoomView.

L'autre point délicat était la porte gardée et verrouillée : deux conditions sur la même sortie. Elles sont séparées dans Exit (locked, guarded) et décidées dans Game.go et Game.use, dans un ordre fixe : d'abord le garde, ensuite la serrure. Les états de la

figure 7 correspondent à cet ordre.

Le jeu reste volontairement court : cinq salles, deux objets utiles, deux personnages, une victoire et une défaite. Cela suffit pour couvrir le sujet et pour que les diagrammes restent lisibles sur une page A4.